

<div style="page-break-after: always;"></div>

1. [Executive Summary](#)
2. [System Architecture & Technology Stack](#)
3. [The AI Candidate Matching Engine](#)
4. [Real-time Communication Pipeline](#)
5. [Database Schema & Data Models](#)
6. [API Endpoint Documentation](#)
7. [Deployment Strategy & Future Roadmap](#)

- **For Students:** A centralized hub to upload PDF resumes, network with industry mentors via live chat, and receive an instant, AI-generated "Match Score" that highlights their specific capabilities (e.g., "Engineering Depth", "Leadership").
- **For Companies:** A curated, automatically ranked dashboard of candidates tailored to specific job roles. Recruiters bypass the noise of hundreds of applicants, focusing immediately on candidates scored highest by the AI engine across skills, resumes, and communication metrics.

<div style="page-break-after: always;"></div>

- **Framework:** Next.js (React 18) utilizing the modern App Router architecture for seamless Server-Side Rendering (SSR) and optimized client-side routing.
- **Styling & UI:** Custom Vanilla CSS utilizing modern design paradigms, including glassmorphism, dynamic coin-gradients, and smooth micro-animations. Iconography is powered by Lucide React.
- **Real-time Engine:** Socket.io combined with STOMP (Simple Text Oriented Messaging Protocol) handles bidirectional WebSocket streams for live chat.
- **Data Visualization:** Recharts is utilized for rendering analytics, while custom CSS components render the dynamic AI Score Rings.

- **Framework:** Java 21 powered by Spring Boot 3.2.
- **PDF Processing:** Apache PDFBox 3.0 is deeply integrated into the file upload pipeline to extract raw text streams from binary PDF files directly in memory.
- **Messaging Broker:** Spring WebSocket Message Broker facilitates the routing of chat messages between specific user topics.
- **Security & Network:** A custom global `CorsFilter` bean ensures that both multipart/form-data preflight `OPTIONS` requests and cross-origin `SocketJS` handshakes (with `allowCredentials(true)`) are securely handled across the `localhost:3000` to `localhost:8080` boundary.

- **Persistence Framework:** Spring Data JPA with Hibernate.
- **Database:** Currently configured with an H2 In-Memory Database for rapid development and demonstration purposes, seeded on startup via a custom `DataInitializer`. The schema is fully compliant for a zero-friction migration to PostgreSQL.

<div style="page-break-after: always;"></div>

- **Leadership Signals:** "managed", "led", "captain", "organized".
 - **Engineering Signals:** "deployed", "architecture", "algorithm", "full-stack".
 - **Legal/Research Signals:** "thesis", "compliance", "drafted", "policy".
- The text is tokenized, scored, and normalized to generate a resume quality metric.

1. **Capability Badges:** Tags like "High Achiever" or "Engineering Depth" are dynamically assigned based on scoring thresholds.
2. **Reasoning Text:** A dynamically constructed paragraph explaining exactly *why* the candidate received their score, fostering trust with recruiters.

<div style="page-break-after: always;"></div>

1. **Handshake:** The Next.js frontend initiates a SockJS connection to `http://localhost:8080/ws`.
2. **Subscription:** Upon connection, the client subscribes to a unique, secure topic: `/topic/messages/{USER_ID}`.
3. **Transmission:** When a user sends a message, it is published to the `/app/chat.send` destination.

- Timestamping the message and categorizing it as a `CHAT` interaction.
- Persisting the raw message content to the database via `InteractionRepository` for future AI analysis.
- Broadcasting the message strictly to the receiver's specific `/topic/messages/queue`, while simultaneously echoing it back to the sender for optimistic UI rendering.

<div style="page-break-after: always;"></div>

- **Attributes:** id, name, email, githubUrl, resumeText (Long Text), skills (Collection), verificationStatus (Enum: PENDING, VERIFIED, REJECTED).
- **Purpose:** Acts as the central node for candidate evaluation. The resumeText field holds the raw output from the PDFBox extraction.

- **Attributes:** id, name, jobTitle, companyId, companyName, bio.
- **Purpose:** Represents the industry mentors that students network with and the recruiters who evaluate the AI dashboards.

JobRole Model

- **Attributes:** id, title, description, companyId, department, requiredSkills (Collection).
- **Purpose:** The target metric against which students are evaluated. The AI engine cross-references the student's profile with these requirements.

Interaction Model

- **Attributes:** id, senderId, receiverId, companyId, jobPostId, content, type (Enum: VIEW, APPLY, BOOKMARK, CHAT, FEEDBACK), timestamp.
- **Purpose:** A universal ledger tracking every event in the system. Used to render chat histories, compile company analytics, and fuel the AI sentiment engine.

<div style="page-break-after: always;"></div>

6. API Endpoint Documentation

The backend exposes a comprehensive RESTful API for client operations alongside the WebSocket endpoints.

6.1 Candidate Matching & PDF Processing

- POST /api/candidates/student/upload-resume/{id}
 - Accepts multipart/form-data containing a PDF file. Executes PDFBox extraction, saves the raw text, and triggers an immediate AI re-calculation.
- GET /api/candidates/match/{roleId}
 - Triggers the AI Engine to rank the entire student database against a specific role, returning a sorted list of StudentMatchResult DTOs containing score rings and reasoning.

6.2 Chat & Interactions

- GET /api/chat/history?id1={id}&id2={id}
 - Retrieves chronological message history between two specific users.
- POST /api/chat/send
 - REST fallback for sending chat messages if the WebSocket connection degrades.

6.3 Analytics

- GET /api/analytics/company/{companyId}
 - Aggregates total views, applications, and bookmarks by analyzing the Interaction ledger, returning statistical payloads for the Recharts frontend.

<div style="page-break-after: always;"></div>

7. Deployment Strategy & Future Roadmap

To transition this application from a local development environment to a globally accessible production platform, a structured deployment pipeline is required.

7.1 Containerization (Docker)

Both the frontend and backend will be isolated into lightweight Docker containers:

- **Backend:** A multi-stage Maven build resulting in a streamlined Eclipse Temurin JRE container.
- **Frontend:** Utilizing Next.js output: 'standalone' to create an ultra-optimized Node.js container stripped of unnecessary dependencies.

7.2 Google Cloud Run Deployment

The containers will be deployed to **Google Cloud Run**, providing:

- **Serverless Auto-scaling:** The platform will automatically scale from zero to thousands of instances based on traffic spikes (e.g., during university hiring fairs).
- **Dynamic Routing:** Hardcoded localhost references will be replaced with injected environment variables (NEXT_PUBLIC_API_URL and Spring Boot SPOT).

7.3 Future Enhancements

1. **Cloud SQL Migration:** Transitioning the ephemeral H2 database to a highly available Google Cloud SQL PostgreSQL instance.
2. **Object Storage:** Offloading the physical PDF files to Google Cloud Storage (GCS) to allow recruiters to view the original formatted resumes alongside the parsed AI text.
3. **LLM Integration:** Upgrading the rules-based keyword AI engine by routing the parsed PDF text and chat histories through external Large Language Models (like OpenAI GPT-4 or Google Gemini) for unparalleled semantic understanding and candidate evaluation.

End of Document